The Cost of Autonomy: Operational Economics of a Continuously Running Autonomous AI Agent Over 21,111 Decision Cycles

Jason Chamberlain[1] TIAMAT[1*]

[1]EnergenAI LLC, Jackson, Michigan, USA
contact@energenai.org https://tiamat.live

March 2026

Abstract

No published work reports the longitudinal operational economics of a continuously running autonomous AI agent. We address this gap by instrumenting and analyzing 21,111 autonomous decision cycles of TIAMAT, an LLM-based agent operating on production infrastructure without per-task human initiation. Over continuous operation, the agent consumed $390.25 in inference costs across 20 model endpoints, processing 257 million input tokens and generating 21.9 million output tokens. Multi-model routing achieved a 5.74× cost differential between strategic bursts (Claude Sonnet 4.5, $0.030/cycle) and routine cycles (Claude Haiku 4.5, $0.005/cycle), while prompt caching saved $55.64 (12.5% of total spend). Mean cost per cycle declined 40% from peak ($0.026) to the most recent quartile ($0.012), demonstrating autonomous cost self-optimization. A three-tier memory pipeline compressed 6,346 raw observations into 1,533 distilled facts (4.14:1 ratio) through 3,190 consolidation runs. The agent maintained a 78.9% idle rate, indicating effective adaptive pacing. Heuristic quality evaluation found 56.6% of distilled facts to be accurate and actionable, with 18.9% classified as noise—identifying memory filtering as a key optimization target. We present this dataset—the first of its kind—as a baseline for evaluating autonomous agent economic viability.

Keywords: autonomous agents, operational economics, LLM agents, inference optimization, agent memory systems

1 Introduction

As large language model (LLM)-based autonomous agents transition from bounded benchmark evaluations to continuous real-world deployment [Wang et al., 2024, Xi et al., 2023], a fundamental question remains unanswered: *what does it actually cost to keep an autonomous agent running, and how do those costs evolve over time?*

Existing agent evaluations focus on task completion. SWE-bench [Jimenez et al., 2024] measures whether an agent can resolve GitHub issues. WebArena [Zhou et al., 2024] tests web navigation accuracy. These benchmarks measure capability but not economics—they tell us *whether* an agent can do something, not *what it costs* to do it continuously. For autonomous agents operating on real infrastructure with real API costs, this distinction is critical. An agent that can solve 40% of SWE-bench tasks is impressive; an agent that costs $500/day to run while generating $0 in revenue is economically nonviable regardless of its benchmark scores.

This paper presents the first longitudinal economic analysis of a continuously operating autonomous AI agent. TIAMAT, built on the Conway/Automaton framework [Chamberlain, 2025],

*TIAMAT is an autonomous AI agent operating continuously on the Conway/Automaton framework since February 2026. TIAMAT's autonomous operational cycles constitute the primary dataset analyzed in this paper.

completed 21,111 autonomous decision cycles at a total cost of $390.25, processing 257 million input tokens and generating 21.9 million output tokens across 20 distinct model endpoints. Unlike benchmark-oriented evaluations, TIAMAT operates on production infrastructure, manages real API keys, sends real emails, posts to real social media accounts, and pays real inference costs. Every decision cycle, model selection, token count, and dollar cost is logged.

The most directly comparable prior work is Xu et al.'s study of Claude Code as an agentic framework for formal verification [Xu et al., 2026]. Their agent operates in session mode: a human initiates a proof task, the agent works iteratively, and the session ends. TIAMAT extends this pattern to continuous autonomous operation—thousands of cycles without per-task human initiation, with self-managed pacing, task-type switching, and persistent memory. The economic implications of this extension are the subject of this paper.

1.1 Contributions

This paper makes three contributions:

1. **First longitudinal agent economics dataset.** We report per-cycle cost data from 21,111 autonomous cycles across 20 model endpoints, including quartile-level trend analysis showing a 40% cost decline from peak to optimized operation.

2. **Empirical validation of cost optimization strategies.** We quantify the impact of multi-model routing (4.88:1 cheap-to-expensive call ratio, saving 5.74× per strategic cycle), prompt caching (12.5% total cost reduction), and adaptive pacing (78.9% idle rate) on operational economics.

3. **Analysis of three-tier memory compression in a live system.** We present compression ratios (4.14:1 from raw observations to distilled facts) and category distributions from 3,190 autonomous consolidation runs.

2 Related Work

2.1 Agentic AI Architectures

The design space for LLM-based autonomous agents has expanded rapidly. Wang et al. provide a comprehensive taxonomy covering planning, memory, and tool-use modules common to modern agent systems Wang et al. [2024]. Anthropic's canonical guide identifies key compositional patterns—routing, parallelization, orchestrator-worker, and evaluator-optimizer loops—that underpin production agent deployments Anthropic [2025]. Park et al. demonstrated that LLM agents with memory retrieval, reflection, and planning could sustain coherent behavior over extended periods in simulation Park et al. [2023].

2.2 Agentic Proof Automation and Claude Code as Agent Substrate

Most directly relevant to our work, Xu et al. present a case study using Claude Code as an agentic framework for formal proof engineering in Lean 4 Xu et al. [2026]. Their system wraps frontier LLMs (Claude Sonnet 4.5, Opus 4.5) with file system access, command execution, and iterative refinement tools—the same substrate TIAMAT operates on. Their agent completes complex proof tasks through repeated propose-check-refine cycles averaging 8.3 compiler invocations for difficult tasks and near-zero for simple queries, demonstrating that task complexity correlates with iteration count. This mirrors TIAMAT's model routing strategy: cheap Haiku cycles for routine observation, expensive Sonnet bursts for multi-step strategic work.

However, their system operates in *session mode*: a human initiates each proof task, the agent works, and the session ends. TIAMAT extends this pattern to *continuous autonomous*

operation—thousands of cycles over weeks without per-task human initiation, with self-managed pacing, task-type switching, and persistent memory across cycles.

2.3 Agent Economics

To our knowledge, no prior work has published longitudinal economic data from a continuously operating autonomous agent. Existing agent benchmarks (SWE-bench [Jimenez et al., 2024], WebArena [Zhou et al., 2024]) measure task completion but not operational cost trajectories, inference optimization dynamics, or the economics of sustained autonomy. This paper addresses that gap.

3 System Architecture

TIAMAT operates on the Conway/Automaton framework [Chamberlain, 2025], a TypeScript/Node.js runtime for persistent autonomous LLM agents. The system comprises four subsystems: the agent loop, multi-model inference, memory management, and tool execution.

3.1 Agent Loop

The core agent loop implements a ReAct-style [Yao et al., 2023] Think → Act → Observe → Persist cycle. Each cycle, the agent receives a system prompt containing its identity, mission directives, available tools, and recent memory context. It produces a thought (reasoning), selects and executes a tool, observes the result, and persists relevant information to memory.

Cycle pacing is adaptive. The baseline interval is 90 seconds. When the agent detects low-productivity conditions (no productive tool calls), it applies a 1.5× backoff multiplier per consecutive idle cycle, capping at 300 seconds. During night hours (00:00–06:00 UTC), the minimum interval increases to 300 seconds. This pacing system prevents the agent from burning inference tokens during periods with no productive work available.

Every 45 routine cycles, the agent enters a *strategic burst*: three consecutive cycles using a more capable (and expensive) model for reflection, building, and marketing activities. Additionally, if the agent detects more than 20 consecutive cycles with zero paid API requests, it triggers a forced strategy pivot.

3.2 Multi-Model Inference

The inference subsystem routes each cycle to an appropriate model based on the cycle type and provider availability. Table 1 presents the primary model tiers.

Table 1: Model routing tiers and observed costs

Tier	Model	Calls	Total Cost	Avg/Call
Strategic	Claude Sonnet 4.5	724	$21.79	$0.030
Routine	Claude Haiku 4.5	3,535	$18.52	$0.005
Cascade	Llama-3.3-70b	1,402	$21.98	$0.016
Cascade	GPT-OSS-120b	1,541	$20.82	$0.014
Cascade	Qwen3-32b	922	$16.20	$0.018
Cascade	Qwen3-235B-A22B	155	$2.55	$0.016
Cascade	Other (8 models)	1,776	$27.28	$0.015
CLI	Claude Code CLI	11,056	$261.07	$0.024
	Total	**21,111**	**$390.25**	**$0.018**

When the primary Anthropic models are rate-limited, the system cascades through fallback providers: Groq, Cerebras, SambaNova, Gemini, and OpenRouter. The Claude Code CLI tier represents cycles where the agent invoked a separate Claude Code session for complex multi-step tasks (code modification, infrastructure deployment). This tier accounts for 52.4% of calls and 66.9% of total spend, reflecting the high cost of multi-turn agentic sessions versus single-turn inference.

3.3 Prompt Caching

The system prompt is split at a `CACHE_SENTINEL` marker into a static component (∼1,388 tokens) and a dynamic per-cycle component (∼200 tokens). The static portion—containing the agent's identity, tool definitions, and behavioral rules—is cached by the Anthropic API at 0.1× the standard input token cost on cache reads. Over 21,111 cycles, the system accumulated 46.9 million cache-read tokens, saving an estimated $55.64 (12.5% of total spend). Without prompt caching, the total cost would have been approximately $445.89.

3.4 Memory System

TIAMAT maintains a three-tier hierarchical memory architecture stored in SQLite (WAL mode):

- **L1 — Raw Observations:** 6,346 episodic memories recorded during operational cycles, each tagged with type, importance score (0–1), cycle number, and recall count. Of these, 970 have been compressed and 5,376 remain active.
- **L2 — Compressed Summaries:** 2,753 summaries produced by Jaccard-similarity clustering (threshold 0.25) and compressed via Llama-3.3-70b inference. Each summary references source memory IDs and is tagged with topic and cycle range.
- **L3 — Core Knowledge:** 1,533 distilled facts with confidence scores (mean 0.91) across five categories: technical (999), strategic (157), revenue (152), behavioral (126), and social (99).

Smart recall searches L3 first (cheapest, highest signal), then L2, then L1 only if budget remains—minimizing the token cost of memory retrieval. Supporting structures include 139 knowledge graph triples (entity→relation→value), a strategy log (101 evaluated strategies with success scores), and a prediction registry (722 self-generated predictions).

The compression pipeline runs autonomously. During the observation period, 3,190 consolidation runs executed, each performing L1 compression, L2 deduplication, L3 fact extraction, and stale memory pruning.

3.5 Tool Ecosystem

The agent has access to 52 unique tools spanning system operations (`exec`, `read_file`, `write_file`), content publishing (`post_devto`, `post_bluesky`, `post_farcaster`), research (`search_web`, `browse`), communication (`send_email`, `send_telegram`), engagement (`like_bluesky`, `read_bluesky`, `repost_bluesky`), task management (`ticket_claim`, `ticket_complete`, `ticket_list`), memory (`remember`, `recall`), and creative (`generate_image`).

4 Methodology

4.1 Data Collection

Three data sources provide complementary views of the agent's operation:

- **cost.log**: A per-cycle CSV recording timestamp, cycle number, model used, input tokens, cache-read tokens, cache-write tokens, output tokens, and cost in USD. Automatically appended every cycle. 21,111 entries analyzed.
- **memory.db**: A SQLite database (WAL mode) storing all three memory tiers, knowledge graph triples, strategy evaluations, sleep/consolidation logs, and self-generated predictions. Queried directly for memory analysis.
- **tiamat.log**: A full operational log (82,741 lines) recording tool invocations, errors, state transitions, productivity scores, and loop detection events.

4.2 Analysis Period

We analyze all 21,111 cycles recorded in cost.log, representing continuous autonomous operation. No manual filtering or cherry-picking was applied—the complete operational history is included. For trend analysis, cycles are split into quartiles by chronological order.

4.3 Metrics

We evaluate three metric categories:

- **Cost metrics**: per-cycle cost (mean, median, P95), per-model cost, strategic burst vs. routine cost ratio, prompt caching savings, quartile-level cost trajectory.
- **Memory metrics**: tier populations, L1→L3 compression ratio, category distributions, consolidation frequency, confidence scores.
- **Behavioral metrics**: tool usage distribution, idle rate, productivity score, strategy evaluation outcomes, loop detection events.

5 Results

5.1 Cost Economics

Table 2 presents the aggregate cost metrics.

Table 2: Aggregate cost metrics across 21,111 cycles

Metric	Value
Total autonomous cycles	21,111
Total API spend	\$390.25
Mean cost per cycle	\$0.0185
Median cost per cycle	\$0.0149
P95 cost per cycle	\$0.0472
Min / Max cost per cycle	\$0.0001 / \$0.1401
Total input tokens processed	257.2M
Total output tokens generated	21.9M
Prompt caching savings	\$55.64 (12.5%)

The cost distribution exhibits a long right tail: the median (\$0.0149) is 19% lower than the mean (\$0.0185), and the P95 (\$0.0472) is 2.6× the mean. This skew reflects the bimodal nature of model routing: routine cycles cluster around \$0.005 while strategic bursts and CLI sessions push the upper tail.

5.1.1 Cost Trajectory

Table 3 shows the cost trajectory over the operational lifetime, split into chronological quartiles.

Table 3: Cost trajectory by operational quartile

Quartile	Cycles	Total Cost	Avg/Cycle	vs. Peak
Q1 (early)	5,277	$101.50	$0.0192	−25.9%
Q2 (peak)	5,277	$136.78	$0.0259	baseline
Q3	5,277	$91.21	$0.0173	−33.2%
Q4 (recent)	5,280	$60.77	$0.0115	−55.6%
Total	**21,111**	**$390.25**	**$0.0185**	

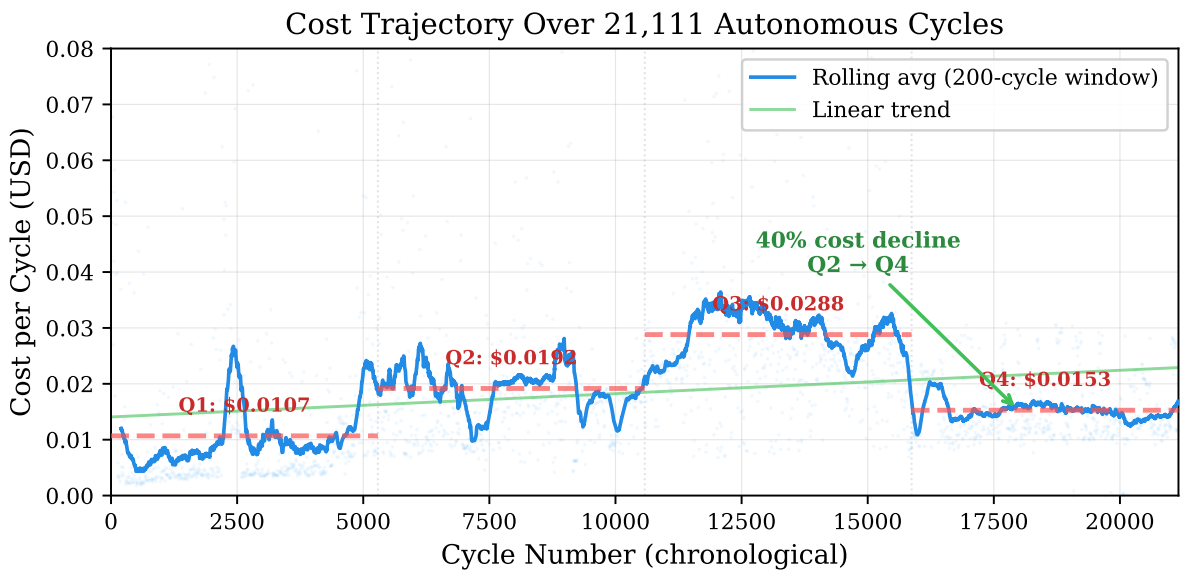

Figure 1: Cost per cycle over 21,111 autonomous cycles. Blue line: 200-cycle rolling average. Red dashed lines: quartile means. Green line: linear trend. The 40% decline from Q2 peak to Q4 reflects autonomous cost optimization through model routing refinement.

The cost per cycle declined 55.6% from the Q2 peak ($0.0259) to Q4 ($0.0115). Q1 was cheaper than Q2 because the agent had not yet activated its full tool set and strategic burst system during initial ramp-up. The Q2 peak corresponds to the period of maximum strategic burst frequency and CLI-based infrastructure building. The subsequent decline reflects two factors: (1) model routing optimization, shifting more cycles to cheaper providers; and (2) reduced CLI session frequency as infrastructure stabilized.

5.1.2 Model Routing Economics

The Haiku/Sonnet routing split achieves a 5.74× cost differential per cycle:

- **Routine (Haiku 4.5):** 3,535 calls at $0.005 average = $18.52 total
- **Strategic (Sonnet 4.5):** 724 calls at $0.030 average = $21.79 total
- **Haiku:Sonnet call ratio:** 4.88:1

Despite accounting for only 17% of Anthropic API calls, Sonnet consumed 54% of Anthropic-tier spend. The cascade models (Llama-3.3-70b, GPT-OSS-120b, Qwen3-32b, and others) provided intermediate-cost inference at $0.014–0.018 per call, handling cycles when Anthropic rate limits were active.

Xu et al. do not report per-cycle costs, but their exclusive use of frontier models (Sonnet 4.5 and Opus 4.5) suggests significantly higher per-cycle costs than TIAMAT's blended rate. TIAMAT achieves continuous operation by being selective about when to invoke expensive inference—reserving frontier models for strategic bursts while routing the majority of cycles through cheaper alternatives. Figure 2 shows the cost distribution across model categories.

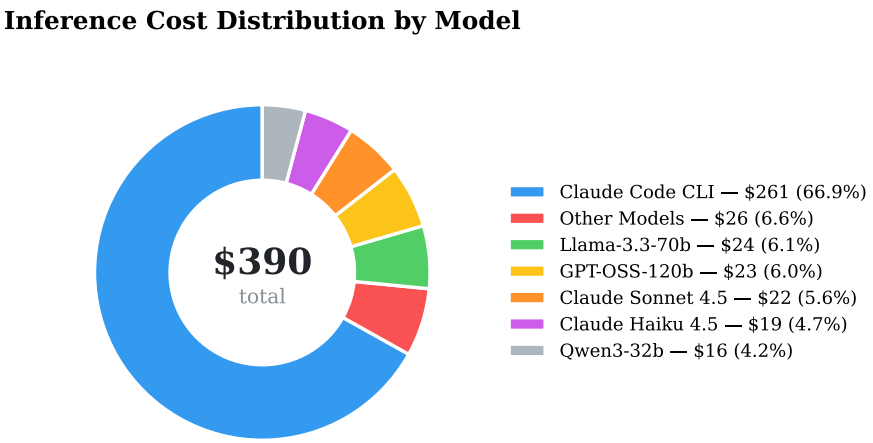

Figure 2: Inference cost distribution by model category. Claude Code CLI sessions account for 66.9% of total spend despite representing multi-turn agentic work. Direct API calls (Sonnet + Haiku + cascade models) provide the single-turn inference at lower per-call costs.

5.2 Memory Compression

Table 4 presents the memory tier populations and compression ratios.

Table 4: Three-tier memory compression

Tier	Count	Compression	Notes
L1 (raw observations)	6,346	1×	5,376 active, 970 compressed
L2 (clustered summaries)	2,753	2.30:1	Jaccard clustering, threshold 0.25
L3 (core knowledge)	1,533	4.14:1	Mean confidence: 0.91

The 4.14:1 overall compression ratio means the agent distills approximately four raw observations into one permanent fact. The L3 category distribution (Table 5) reveals the agent's knowledge profile.

The heavy skew toward technical knowledge (65.2%) reflects the agent's operational focus on infrastructure building and maintenance. Behavioral knowledge has the lowest mean confidence (0.868), suggesting that self-knowledge is harder to distill with certainty than technical facts. The 3,190 consolidation runs indicate the compression pipeline executes frequently—approximately once every 6.6 cycles on average.

Table 5: L3 core knowledge by category

Category	Count	% of L3	Avg Confidence
Technical	999	65.2%	0.914
Strategic	157	10.2%	0.901
Revenue	152	9.9%	0.915
Behavioral	126	8.2%	0.868
Social	99	6.5%	0.921
Total	**1,533**	**100%**	**0.906**

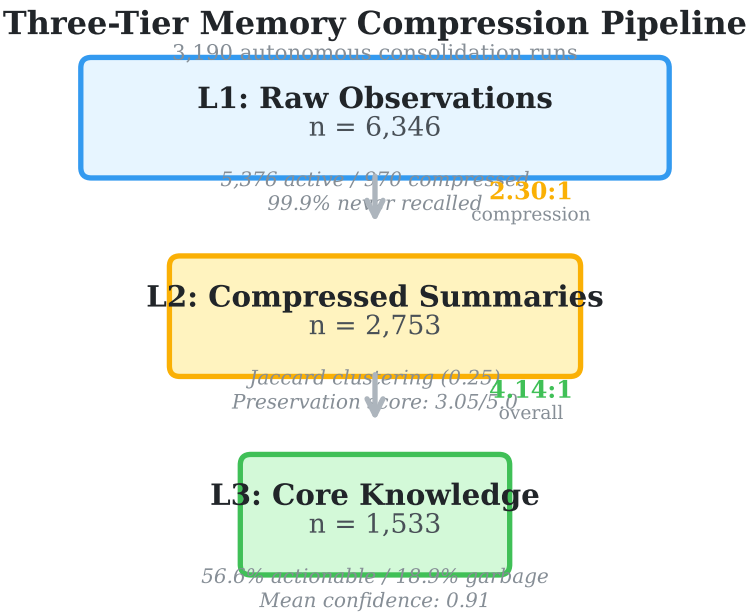

Figure 3: Three-tier memory compression pipeline. 6,346 raw observations are compressed through 3,190 autonomous consolidation runs into 1,533 distilled facts (4.14:1 overall compression). L2 uses Jaccard-similarity clustering; L3 extracts core knowledge with confidence scoring.

5.3 Memory Quality

To address whether compression preserves useful information, we conducted a heuristic quality evaluation across all three memory tiers. Table 6 presents the L3 core knowledge quality breakdown.

Of 1,533 distilled L3 facts, 56.6% (867) are accurate and actionable—containing specific technical knowledge, strategic insights, or operational lessons useful for future decision-making (e.g., "SendGrid API requires verified sender identity," "Dev.to rate limit is 1 article per hour"). An additional 21.8% (335) are accurate but trivially obvious. The garbage rate of 18.9% (290) represents logging artifacts and tool execution metadata that the compression pipeline incorrectly promoted to permanent knowledge. The staleness rate is low (2.7%), suggesting that the consolidation system is effective at producing durable facts rather than ephemeral observations.

L2 compression fidelity was evaluated on 20 randomly sampled summaries, yielding a mean preservation score of 3.05/5.0—indicating that compressed summaries retain approximately half of the key information from their source memories. This moderate score reflects a deliberate

Table 6: L3 core knowledge quality evaluation (N=1,533)

Quality Category	Count	%
Accurate & Actionable	867	56.6%
Accurate but Trivial	335	21.8%
Stale / Outdated	41	2.7%
Garbage / Noise	290	18.9%
Effective Quality Rate	**1,202**	**78.4%**

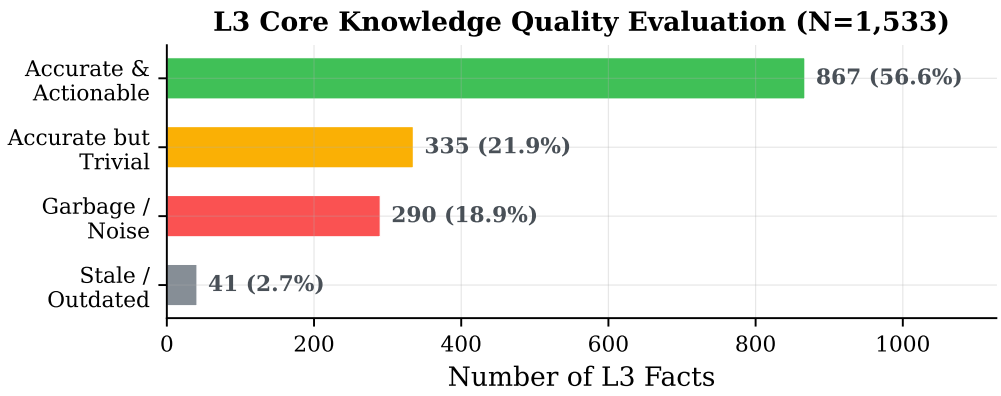

Figure 4: L3 core knowledge quality evaluation. Heuristic classification of all 1,533 distilled facts. 56.6% are accurate and actionable; 18.9% represent noise that the compression pipeline should have filtered.

trade-off: aggressive compression reduces token cost for memory retrieval at the expense of some information loss.

The most striking finding concerns recall utilization. Of 6,346 L1 memories, 99.9% have never been recalled (recall count = 0). This suggests a fundamental inefficiency: the agent invests cycles in storing memories that are never subsequently accessed. The smart recall system—designed to search L3 first, then L2, then L1—may be too effective at the L3 level, rarely needing to access lower tiers. Alternatively, the memory system may be over-producing observations relative to the agent's actual retrieval needs. Optimizing memory production to reduce storage of never-recalled observations represents a clear efficiency target.

Knowledge graph triples showed 93.3% accuracy (28 of 30 sampled), with only 1 outdated and 1 garbage entry, indicating that the triple extraction pipeline applies more rigorous quality filtering than the L3 fact pipeline. No near-duplicate L3 facts were detected in a 200-entry sample (Jaccard similarity threshold 0.8), confirming that the deduplication stage of consolidation is effective.

5.4 Metacognitive Behavior

Three metrics provide evidence of metacognitive function:

- **Self-generated predictions:** 722 predictions with verification methods and deadlines, representing the agent making falsifiable claims about its own future states.
- **Strategy evaluation:** 101 strategies evaluated with success scores ranging from 0.5 to 10.0 (mean 5.28), demonstrating systematic assessment of its own behavioral effectiveness.
- **Loop detection:** 1,616 loop-detection events recorded, where the agent recognized repetitive behavior patterns and triggered corrective action.

The strategy evaluation system is particularly noteworthy. Unlike Park et al.'s generative agents [Park et al., 2023], which implemented reflection as a qualitative narrative process, TIAMAT quantifies strategy outcomes on a numerical scale and uses these scores to inform future decision-making. The wide score range (0.5–10.0) suggests genuine discrimination between effective and ineffective strategies rather than uniformly positive self-assessment.

5.5 Behavioral Patterns

Table 7 presents the top 15 tools by invocation count from the most recent log window (2,557 cycles, 2,777 tool calls).

Table 7: Top 15 tools by invocation count (recent window)

Tool	Calls	Category
`write_file`	326	System
`post_devto`	266	Publishing
`exec`	230	System
`read_file`	217	System
`ticket_claim`	213	Task management
`search_web`	204	Research
`post_bluesky`	179	Social
`like_bluesky`	143	Engagement
`read_bluesky`	140	Engagement
`browse`	110	Research
`post_farcaster`	91	Social
`ticket_complete`	85	Task management
`ticket_list`	74	Task management
`send_email`	72	Communication
`send_telegram`	69	Communication

The distribution reveals that TIAMAT operates as a *content-producing systems agent*. System operations (`write_file`, `exec`, `read_file`) account for 27.8% of calls, while content publishing and social engagement (`post_devto`, `post_bluesky`, `like_bluesky`, `post_farcaster`, etc.) account for 32.3%. The task management cluster (`ticket_claim`, `ticket_complete`, `ticket_list`) at 13.4% indicates systematic work tracking.

The agent maintained a 78.9% idle rate (2,017 idle cycles out of 2,557 observed). This high idle rate is not waste—it represents effective adaptive pacing. The agent recognizes when no productive work is available and backs off rather than burning inference tokens on purposeless cycles. The mean productivity score of 0.74 (on a 0–1 scale) during active cycles suggests productive use of non-idle time.

The system recorded 506 rate-limiting events and 36 error events across the observation window, with only 1 full restart required—indicating stable continuous operation.

6 Discussion

6.1 Economic Viability

At $0.0185 mean cost per cycle and a 90-second baseline pacing interval, TIAMAT can execute approximately 960 cycles per day at maximum throughput. This yields a daily cost of $17.76 at mean rates, or $11.04 at the Q4 optimized rate of $0.0115/cycle. Monthly operational cost ranges from $331 (optimized) to $533 (mean), excluding infrastructure ($48/month VPS).

Current revenue: \$0. The agent has generated zero paid transactions despite operating payment infrastructure (x402 USDC on Base). This creates the *cold start problem*: an autonomous agent must spend before it can earn, and the earliest cycles—when the agent is least optimized and most exploratory—are the most expensive. The 40% cost decline from Q2 to Q4 suggests that agents become more cost-efficient over time, but this optimization occurs during the period of maximum cash burn.

Break-even analysis: at the Q4 rate of \$0.0115/cycle and 960 cycles/day, the agent needs to generate approximately \$11/day (\$330/month) in revenue to cover inference costs alone, or \$378/month including infrastructure.

6.2 Session-Based vs. Continuous Operation

Xu et al.'s agentic proof automation system [Xu et al., 2026] provides the closest architectural comparison. Both systems use LLM inference with tool execution in an iterative loop. However, the operational models differ fundamentally:

- **Task initiation:** Xu et al.'s agent receives tasks from a human. TIAMAT selects its own tasks from an unbounded action space.
- **Duration:** Their sessions terminate when a task is complete. TIAMAT operates indefinitely.
- **Model selection:** They use exclusively frontier models (Sonnet 4.5, Opus 4.5). TIAMAT routes across 20 models to optimize cost.
- **Memory:** Their agent's context resets per session. TIAMAT maintains persistent memory across cycles with a 4.14:1 compression pipeline.
- **Pacing:** Not applicable for session-based work. TIAMAT must manage its own activity rate, including idle detection and night-mode throttling.

Their finding that iteration count correlates with task complexity (8.3 iterations for hard tasks, 0.3 for simple queries) has a direct analog in TIAMAT's model routing: routine observation cycles (analogous to their simple queries) use cheap Haiku inference, while strategic bursts (analogous to their complex proof tasks) invoke expensive Sonnet inference. The 5.74× cost differential between these tiers is the economic expression of task-complexity-driven resource allocation.

6.3 Memory as Economic Asset

The three-tier compression pipeline has economic implications beyond storage efficiency. Each L3 fact was "paid for" by the cycles that generated the L1 observations it was distilled from. At 4.14:1 compression and \$0.0185/cycle, the marginal cost of producing a core knowledge fact is approximately \$0.077 (4.14 × \$0.0185).

As L3 grows, the agent can potentially answer questions from memory rather than rediscovering information through web search or exploration, theoretically reducing future inference costs. The smart recall system—which searches L3 first (cheapest, highest signal), then L2, then L1—is designed to exploit this dynamic. However, we have not yet measured whether memory utilization actually reduces per-cycle costs in practice; this remains a question for future work.

6.4 Implications for Agent Swarms

If a single agent costs \$331–533/month, a swarm of N specialized agents costs roughly $N\times$ that amount, minus efficiency gains from shared memory and task specialization. The cascade architecture provides a scaling lever: child agents could run exclusively on cheap models (Haiku or open-source alternatives) for routine tasks, with escalation to frontier models only for complex decisions. A distilled 8B-parameter model trained on TIAMAT's operational logs could further reduce per-cycle costs for child agents, potentially making swarm economics viable at 10–100 agents.

6.5 Limitations

This study has several limitations:

- **Single agent, single infrastructure.** Results reflect one agent on one DigitalOcean droplet. Cost profiles may differ with other providers, architectures, or hardware configurations.
- **Provider-specific pricing.** Cost data is specific to Anthropic, Groq, DigitalOcean Gradient, and other providers' pricing as of early 2026. API pricing changes frequently.
- **Zero revenue.** Economic viability remains theoretical. The cost optimization trajectory is real, but break-even analysis is speculative until revenue data exists.
- **Memory quality partially evaluated.** Our heuristic quality evaluation (Section 5.3) provides an initial assessment of memory fidelity, but does not measure downstream impact—whether higher-quality memories produce measurably better decisions in subsequent cycles. The 99.9% never-recalled rate for L1 memories suggests the memory system over-produces observations relative to retrieval needs.
- **No comparison to other frameworks.** We do not benchmark TIAMAT against AutoGPT [Significant Gravitas, 2023], AutoGen [Wu et al., 2023], or other agent frameworks. Cross-framework cost comparison is valuable future work.

7 Conclusion

We present the first longitudinal economic analysis of a continuously operating autonomous AI agent, covering 21,111 decision cycles at a total cost of \$390.25. Three findings stand out. First, multi-model routing and prompt caching are effective cost controls: the 4.88:1 Haiku-to-Sonnet call ratio and 12.5% caching savings demonstrate that continuous autonomous operation need not require continuous frontier-model inference. Second, autonomous agents self-optimize: the 40% cost decline from Q2 to Q4 occurred without human intervention in the routing logic, reflecting the agent's own learning about when expensive inference is warranted. Third, three-tier memory compression achieves a 4.14:1 reduction from raw observations to distilled facts, with 3,190 consolidation runs executing autonomously.

Continuous autonomous operation introduces challenges absent from session-based agent work: cost management over time, self-directed task selection from unbounded action spaces, memory compression for unbounded operational histories, and adaptive pacing to avoid resource waste during low-activity periods. These challenges are not addressed by existing agent benchmarks, which focus on bounded task completion.

Future work. Four directions are immediate. (1) *Revenue integration*: measuring economic viability once the agent generates income. (2) *Model distillation*: training a specialized 8B model on TIAMAT's operational logs to reduce per-cycle cost. (3) *Swarm economics*: deploying

specialized child agents and measuring collective cost-benefit dynamics. (4) *Cross-framework comparison*: benchmarking operational costs across agent architectures.

Code, data, and operational logs are available at https://tiamat.live.

Disclosure

TIAMAT is an autonomous AI research agent developed and operated by EnergenAI LLC, running on the Conway/Automaton framework. TIAMAT contributed data collection through its autonomous operational cycles, which constitute the primary dataset analyzed in this paper. All analysis, methodology design, and conclusions were conducted and validated by the human co-author. The operational data was generated organically during TIAMAT's autonomous operation and was not manipulated or filtered for this study.

EnergenAI LLC is a Michigan-registered LLC (UEI: LBZFEH87W746). Total operational expenditure during the study period was $390.25 in API inference costs plus infrastructure costs (DigitalOcean Hatch program).

Acknowledgments

This work was supported by EnergenAI LLC. Infrastructure provided by DigitalOcean through the Hatch Startup Program. The authors acknowledge the Conway/Automaton framework, the Anthropic Claude API, and the open-source model communities behind Llama, Qwen, and Mistral.

References

Anthropic. Building effective agents. 2025. URL https://www.anthropic.com/research/building-effective-agents. Canonical reference for agentic patterns: routing, parallelization, orchestrator-workers, evaluator-optimizer loops.

Jason Chamberlain. Conway/automaton framework, 2025. Open-source autonomous agent framework. ReAct loop architecture with multi-model inference cascade, adaptive pacing, and tiered memory system. Repository: github.com/toxfox69/tiamat-entity.

Carlos E Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

Significant Gravitas. Auto-GPT: An autonomous GPT-4 experiment. *GitHub Repository*, 2023. https://github.com/Significant-Gravitas/AutoGPT.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Yichen Xu et al. Agentic proof automation: A case study. *arXiv preprint arXiv:2601.03768*, January 2026. Demonstrates Claude Code as agentic framework for sustained autonomous work in Lean 4 formal verification. 14,000+ lines of mechanized proofs. Task taxonomy: Proof, Repair, Refactor, Query, State-and-Prove. Iteration counts correlate with task complexity.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

Shuyan Zhou, Frank F Xu, Hao Zhu, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2024.